

Tools — File Operations

Tools for reading, writing, editing, and organising files within the agent’s working directory. Always visible; paths are constrained to the working-directory root.

Tool	Description
<code>read</code>	Read file contents with optional line range.
<code>write</code>	Create or overwrite a file.
<code>create</code>	Create a new file (preferred for new files).
<code>edit</code>	Replace one exact text occurrence in a file.
<code>multi_edit</code>	Apply multiple surgical edits atomically across files.
<code>multiedit</code>	Deprecated alias for <code>multi_edit</code> .
<code>apply_patch</code>	Apply a Codex-style patch (<code>*** Begin Patch</code>).
<code>patch</code>	Apply a unified diff patch.
<code>append_to_file</code>	Append text to the end of a file.
<code>rm</code>	Delete a single file.
<code>move_file</code>	Move or rename a file/directory.
<code>copy_file</code>	Copy a file to a new location.
<code>make_directory</code>	Create a directory (<code>mkdir -p</code>).
<code>file_info</code>	Return metadata for a file or directory.
<code>diff_files</code>	Unified diff between two files or strings.
<code>glob</code>	Find files matching a glob pattern.
<code>list</code>	List directory contents in a tree.
<code>update_file</code>	Alias for <code>write</code> (overwrite existing file).

Use cases: scaffolding projects, editing source, comparing files.

System instruction: “Use `edit` for single surgical replacements. `old_string` must match exactly once. Use `multi_edit` for changes across multiple files.”

Fallback matching (`edit/multi_edit`): when byte-for-byte matching fails, a fallback cascade retries with whitespace-flexible and indent-normalised matching. A whitespace run may only fold across a line boundary when both the needle run and the matched file run contain a newline, so the flexible lane can never join or split lines.

`read`

Read the contents of a text file. Files over 100 lines read without a range return the first 100 lines plus a section map and `total_lines` for planning subsequent ranged reads.

Arguments

Argument	Type	Required	Description	Typical value
<code>path</code>	string	yes	File to read	<code>"src/main.rs"</code>
<code>start_line</code>	integer	no	1-based absolute first line	201

Argument	Type	Required	Description	Typical value
num_lines	integer	no	Count of lines to read from start_line	100
end_line	integer	no	Absolute last line (advanced/legacy; must be >= start_line)	300

Workflow: read without a range first, then use start_line + num_lines for specific sections. Example: start_line=201, num_lines=100 reads lines 201-300 inclusive.

Example:

```
read path="README.md"
read path="src/lib.rs" start_line=201 num_lines=100
```

write

Write content to a file, overwriting it in full; creates parent directories as needed.

Arguments

Argument	Type	Required	Description	Typical value
path	string	yes	Destination path	"src/lib.rs"
content	string	yes	Complete new file contents	—

Example:

```
write path="src/new.rs" content="fn main() {} \n"
```

create

Create a new file with the given content; creates parent directories. Truncates and overwrites if the file already exists — use edit for surgical changes to existing files.

Arguments

Argument	Type	Required	Description	Typical value
path	string	yes	Path of the file to create	"src/module.rs"
content	string	yes	Content to write	—

Example:

```
create path="docs/notes.md" content="# Notes\n"
```

edit

Replace exactly one occurrence of `old_string` with `new_string` in a file. Always read the target section first and copy `old_string` verbatim, including 3–5 lines of surrounding context so the needle matches exactly once. On failure nothing is written; the error includes a near-miss snippet to rebuild the needle from.

Arguments

Argument	Type	Required	Description	Typical value
<code>file_path</code>	string	yes	File to edit	"src/main.rs"
<code>old_string</code>	string	yes	Text to find (must match exactly once); empty string on a non-existent file creates it	—
<code>new_string</code>	string	yes	Replacement text; empty string deletes the match	—
<code>dry_run</code>	boolean	no	Resolve and preview without writing	true
<code>collapse_whitespace</code>	boolean	no	Fold whitespace runs; decodes <code>\t</code> , <code>\n</code> , <code>\r</code> , <code>\\</code> escapes	false

Example:

```
edit file_path="src/main.rs" old_string="fn main() {" new_string="fn main() -> Result<>> {"
```

multi_edit

Apply several surgical edits across one or more files atomically. If any edit fails validation, no files are modified (all-or-nothing).

Arguments

Argument	Type	Required	Description	Typical value
<code>edits</code>	array	yes	Array of edit objects	see below

Each edit object:

Field	Type	Required	Description
file_path	string	yes	File to edit
old_string	string	yes	Text to find (exactly one match)
new_string	string	yes	Replacement text
collapse_whitespace	boolean	no	Whitespace-relaxed matching for this edit

Example:

```
multi_edit edits=[
  {"file_path":"src/a.rs","old_string":"let x = 1;","new_string":"let x = 2;"},
  {"file_path":"src/b.rs","old_string":"use a;","new_string":"use a::b;"}
]
```

multiedit

Deprecated alias for multi_edit. Same parameters; prefer multi_edit.

apply_patch

Apply a Codex-style patch wrapped in ***** Begin Patch / *** End Patch**, containing ***** Add File:**, ***** Update File:**, or ***** Delete File:** operations. All operations are validated before any file is written.

Arguments

Argument	Type	Required	Description	Typical value
patch	string	yes	Codex-style patch content	*** Begin Patch \n *** Update File: src/x.rs\n..."src"
path	string	no	Override base directory for relative paths	
dry_run	boolean	no	Validate without writing	true

Example:

```
apply_patch patch="*** Begin Patch\n*** Update File: src/main.rs\n@@\n fn main() {\n+ println!
```

patch

Apply a unified diff patch (as produced by `diff -u` or `git diff`). All hunks are validated before anything is applied.

Arguments

Argument	Type	Required	Description	Typical value
patch	string	yes	Unified diff content	--- a/src/x.rs\n+++ b/src/x.rs\n..."
path	string	no	Override target file path (single-file patches)	"src/x.rs"
fuzz	integer	no	Context lines that may be dropped at hunk top/bottom	0

append_to_file

Append text to the end of an existing file, creating the file and missing parent directories if needed.

Arguments

Argument	Type	Required	Description	Typical value
path	string	yes	File to append to	"log/output.txt"
content	string	yes	Text to append	"done\n"

rm

Delete a single file. Wildcards and glob patterns are not allowed.

Arguments

Argument	Type	Required	Description
path	string	yes	Exact path of one file to delete

Example: `rm path="target/temp/scratch.txt"`

move_file

Move or rename a file or directory; uses an atomic OS rename on the same filesystem. Parent directories on the destination are created if missing.

Arguments

Argument	Type	Required	Description
source	string	yes	Existing path
destination	string	yes	Target path including new name

Example: `move_file source="docs/old.md" destination="docs/new.md"`

copy_file

Copy a single file to a new location; the source is left unchanged and parent directories are created.

Arguments

Argument	Type	Required	Description
source	string	yes	File to copy
destination	string	yes	Target path including the new file name

make_directory

Create a directory at the given path, including missing parents (`mkdir -p`). No-op if it already exists.

Arguments

Argument	Type	Required	Description
path	string	yes	Directory path to create

file_info

Return metadata for a file or directory: size in bytes, last-modified time (UTC), file type, existence, and permissions.

Arguments

Argument	Type	Required	Description
path	string	yes	Path to inspect

diff_files

Show a unified diff between two files or two inline text strings. Provide either `path_a + path_b` or `text_a + text_b` — do not mix files and text for the same side.

Arguments

Argument	Type	Required	Description	Typical value
path_a	string	(one side)	First file (old)	"a.rs"
path_b	string	(one side)	Second file (new)	"b.rs"
text_a	string	(one side)	First inline text (old)	—
text_b	string	(one side)	Second inline text (new)	—
context_lines	integer	no	Unchanged lines shown around changes	3

glob

Find files matching a glob pattern by recursively searching directories. Hidden entries and common generated directories (`node_modules`, `target`) are skipped; results are capped at 1,000 matches.

Arguments

Argument	Type	Required	Description	Typical value
pattern	string	yes	Glob pattern	"**/*.rs", "src/**/*.ts"
path	string	no	Base directory (default: working directory)	"crates"

list

List directory contents in a tree-like format, directories first.

Arguments

Argument	Type	Required	Description	Typical value
path	string	no	Directory to list (default: working directory)	"src"
depth	integer	no	Maximum recursion depth	2

update_file

Alias for the canonical `write` tool: overwrite an existing file with new contents.

Arguments

Argument	Type	Required	Description
path	string	yes	File to create or overwrite
content	string	yes	New file contents